

Module M05.C03 — SQL pour transformer : `CREATE TABLE AS`, `WITH`, `TRY\_CAST`, et la jointure à la clé complète

Table des matières

MODULE M05.C03 — SQL POUR TRANSFORMER : CREATE TABLE AS, WITH, TRY_CAST, ET LA JOINTURE À LA CLÉ COMPLÈTE	3
1. Objectifs du chapitre	3
2. Pourquoi cette notion est importante	3
3. Explication simple	4
4. Vocabulaire essentiel	4
5. Cours approfondi — la table des 10 opérations en SQL DuckDB	6
5.1 Session en mémoire et chargement du CSV	6
5.2 Étape 1 — Sélectionner / renommer les colonnes	6
5.3 Étape 2 — Convertir le type (avec compteur d'échec)	7
5.4 Étape 3 — Nettoyer le texte	7
5.5 Étape 4 — Filtrer les lignes (la coupe à la date)	8
5.6 Étape 5 — Trier	8
5.7 Étape 6 — Dédoublonner (par rang)	8
5.8 Étape 7 — Colonne calculée (la transposition jour/mois)	9
5.9 Étape 8 — Fusionner (la jointure aux remises)	9
5.10 Étape 9 — Agréger (CA par catégorie)	10
5.11 Étape 10 — Pivoter / dépivoter en SQL	10
5.12 Étape 11 — Exporter la table propre	11
5.13 La session en mémoire, persistée seulement à l'export	11
5.14 Le diagnostic en 4 questions (le « Health Check » SQL)	11
6. Exemple concret — la requête mars2025.sql complète	12
7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table	15
7.1 La sérialisation canonique en SQL	15
7.2 Le calcul de l'empreinte en Python	16
7.3 Le verdict du projet M05.P (la mesure d'ouverture)	16
7.4 La rejouabilité : avril et mai 2025	16
8. Erreurs fréquentes	17
9. Bonnes pratiques professionnelles	17
10. Exercice guidé — la requête avril2025.sql paramétrée (45 min, /10)	18
11. Exercices autonomes	18
12. Correction détaillée	19
13. Mini-projet M05.P3 — « La requête SQL du projet M05.P » (1 h 30)	19
14. Boîte à outils du chapitre	20
15. Résumé du chapitre	20
16. À retenir	21
17. Évaluation formative (auto-correction, 8 min)	21

Module M05.C03 — SQL pour transformer : `CREATE TABLE AS`, `WITH`, `TRY_CAST`, et la jointure à la clé complète

Outil de ce chapitre : SQL DuckDB 1,5,5 (en mémoire), sur le fil rouge mars 2025 du socle (6 884 lignes). Durée indicative : 7 h. Niveau : N2.

L'idée du chapitre. SQL n'est pas un outil de consultation : c'est un outil de **transformation**. Les 10 opérations du C01 (sélectionner, convertir, nettoyer, filtrer, trier, dédoublonner, calculer, fusionner, agréger, pivoter/dépivoter) ont toutes une forme SQL — et **DuckDB 1,5,5 les exécute en mémoire**, sans base à installer. Le chapitre installe la **session en mémoire** (règle §1.3 du plan M05 : pas de `.duckdb` persisté), les trois formes de création de table (`CREATE TABLE AS`, `INSERT...SELECT`, `WITH`), le nettoyage **dans la requête** (`TRY_CAST`, `TRIM`, `REPLACE`, le dédoublonage par rang), et la **jointure à la clé complète** contre la **jointure naïve** (la leçon du module : 3 978 lignes au lieu de 101 sur mars, sans message d'erreur).

Base de travail — le fil rouge mars 2025, mesuré en DuckDB 1,5,5. Coupe à la date 2025-03-01 → 2025-03-31 : **6 884 lignes · total montant_ttc 464 096 003 FCFA** (mesure `m05_filrouge_mars_total_ttc`). La requête complète du §6 produit **la même table** que la requête Power Query du C02 (les deux ont été réécrites pour appliquer les mêmes 14 opérations dans l'ordre canonique du C01) — c'est le **verdict** : trois moteurs, une table, une empreinte sha256 (mesure `m05p_empreinte_sha256` = 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465).

1. Objectifs du chapitre

À la fin de ce chapitre, vous saurez :

- **créer une session DuckDB en mémoire** et y charger un CSV par `read_csv_auto` (règle §1.3 du plan M05 : pas de `.duckdb` persisté) ;
- **créer une table transformée** par `CREATE TABLE AS`, `INSERT...SELECT` ou `WITH` ;
- **convertir en SQL** avec `TRY_CAST` + compteur d'échec (`COUNT(*) WHERE ... IS NULL`) ;
- **nettoyer en SQL** avec `TRIM`, `REPLACE`, `STRPTIME`, `STRftime` ;
- **dédoublonner en SQL** par rang (`ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`) — l'unique méthode qui marche dans tous les moteurs (DuckDB, PostgreSQL, BigQuery, Snowflake, Redshift) ;
- **fusionner à la clé complète** (`id_client × annee × mois`) et **comparer** avec la jointure naïve (`id_client` seul) — la leçon du module (3 978 vs 101 sur mars, sans message d'erreur) ;
- **agréger et pivoter en SQL** (`GROUP BY`, `SUM(CASE WHEN ... THEN ... END)`) ;
- **charger la table propre** en CSV ou Parquet via `COPY (SELECT ... FROM ...) TO '...' ;`
- **vérifier le verdict** par empreinte sha256 — trois moteurs, une table.

2. Pourquoi cette notion est importante

- SQL est l'outil de **transformation en volume** : Power Query est limité à $\approx 500\,000$ lignes (règle du C02), pandas commence à ralentir au-delà du million. DuckDB, lui, monte jusqu'à 10-50 millions de lignes sans suer — et reste **30 × plus rapide** que pandas sur ce fichier (0,4 s contre 3,1 s sur 6 884 lignes, mesure indicative à vérifier en C05).

- La **jointure à la clé complète** est la règle d'or du module : sans elle, la jointure naïve rend **3 978 lignes** au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`, $\approx 39 \times$ trop), et **41 716 lignes** au lieu de 1 714 sur la fenêtre du projet (mesure `m05_fenetre_jointure_naive`, $\approx 24 \times$ trop). DuckDB ne dit rien — la requête tourne, le résultat est faux. La règle est toujours la clé complète, et on la documente dans un commentaire au-dessus de la jointure.
- Le **dédoublonnage par rang** (`ROW_NUMBER() OVER ...`) est la méthode universellement supportée — PostgreSQL, BigQuery, Snowflake, Redshift, DuckDB — alors que `DISTINCT ON` n'existe qu'en PostgreSQL et `QUALIFY` n'existe qu'en BigQuery/Snowflake. Apprendre la méthode `ROW_NUMBER()`, c'est pouvoir dédoubler partout.
- Le **compteur d'échec** est la première discipline (règle 3 du §3 du plan M05). DuckDB `read_csv_auto` est tolérant : il avale les 1 710 montants texte sans un mot et les met en `VARCHAR`. Sans compteur d'échec, on additionne du texte et on perd 1,5 % du CA sans s'en apercevoir.
- La **session en mémoire** (règle §1.3 du plan M05) est ce qui rend SQL aussi léger que pandas : pas de serveur à installer, pas de fichier `.duckdb` à nettoyer, pas de mot de passe. Le pipeline est jetable : on perd la session à la fin du notebook. C'est aussi un risque (la session n'est pas reproductible) — d'où l'**export CSV/Parquet** en fin de pipeline, qui rend la table propre persistante et transmissible.
- Enfin, **SQL n'est pas un langage de script** : c'est un langage de **transformation de tables**. La tentation du débutant est de faire des boucles (`FOR ... IN ... LOOP`) : DuckDB ne supporte pas les boucles SQL natives, et c'est **une vertu** (les boucles sont lentes, illisibles, et remplaçables par des jointures). Le chapitre le démontre.

3. Explication simple

On ne programme pas en SQL : on **décrit une transformation de table**. La requête prend une table (source), en produit une autre (résultat), et DuckDB optimise l'exécution. Le résultat est une table qu'on peut utiliser comme source d'une autre requête, ou exporter en CSV/Parquet.

La requête complète du fil rouge (mars 2025) a **6 blocs** dans C03 : `WITH src AS (... conversion ...)` → `coupe` (filtre à la date) → `cle` (dédoublonnage par rang) → `dedup` (filtre du rang 1) → `repare` (transposition jour/mois) → `enrichi` (jointure aux remises). C'est **moins long** que la requête Power Query de C02 (30 lignes de code M) — et exécutable dans l'atelier.

La **règle d'or** du SQL DuckDB : **chaque table temporaire a un nom, en toutes lettres, qui dit ce qu'elle contient**. Une table temporaire sans nom (`t1`, `t2`) est une table que personne ne peut relire.

4. Vocabulaire essentiel

Français — English	Définition simple	Piège à éviter
Session en mémoire — In-memory session	Une base DuckDB qui vit dans la RAM du processus Python (ou du client CLI). Pas de fichier <code>.duckdb</code> à créer, à nettoyer, à versionner. La session meurt à la fin du processus.	Persister la base dans un fichier <code>.duckdb</code> : c'est plus lourd, et le fichier se désynchronise de la source. Règle §1.3 du plan M05 : pas de base persistée.
<code>read_csv_auto</code>	La fonction DuckDB qui lit un CSV en inférant les types (équivalent de <code>pd.read_csv</code> avec <code>dtype</code> déduit). Rapide (0,4 s sur 6 884 lignes), tolérant (avale les 1 710 montants texte sans erreur).	Se fier au type déduit : <code>read_csv_auto</code> met <code>montant_ttc</code> en <code>VARCHAR</code> , et les sommes sautent ces lignes en silence. Règle du module : toujours reconverter avec <code>TRY_CAST</code> et compter les échecs.

Français — English	Définition simple	Piège à éviter
CREATE TABLE AS	La forme canonique de création d'une table à partir d'une requête. Plus rapide que INSERT...SELECT (un seul parcours), atomique (la table n'existe pas pendant l'exécution).	Oublier le CREATE OR REPLACE TABLE : la requête échoue si la table existe déjà (« Table already exists »).
INSERT...SELECT	Insertion de lignes depuis une requête dans une table existante. Utile pour les tables cibles pré-crées (les INSERT sont plus traçables dans le lineage).	Faire un INSERT sans WHERE : on duplique les lignes à chaque exécution. Règle : toujours vérifier l'unicité de la clé.
WITH	Une Common Table Expression (CTE) : une sous-requête nommée, réutilisable dans la requête principale. C'est l'équivalent des étapes Power Query.	Enchaîner 15 WITH sans nommer clairement : illisible. La règle est un WITH par étape conceptuelle, nommé en français.
TRY_CAST	Conversion de type qui renvoie NULL (DuckDB) en cas d'échec, sans erreur. L'alternative stricte est CAST (qui fait échouer toute la ligne).	Utiliser CAST sur des données sales : une cellule « 1 200 FCFA » fait échouer toute la requête. Règle : toujours TRY_CAST + compteur COUNT(*) WHERE ... IS NULL.
STRPTIME STRftime	/ Conversion string ↔ date. STRPTIME parse, STRftime formate. Le format '%Y-%m-%d' est l'ISO 8601 (le format du fichier livré).	Confondre les formats : '%d/%m/%Y' (français) et '%m/%d/%Y' (américain) sont inversés — DuckDB ne devine pas, il faut le préciser.
ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)	Attribue un rang entier à chaque ligne d'un groupe (la PARTITION BY), triée selon ORDER BY. C'est l'outil du dédoublonnage par rang.	Oublier le WHERE rn = 1 : toutes les lignes sont renvoyées, le dédoublonnage n'a pas eu lieu.
COUNT(*) WHERE ... IS NULL	Compteur d'échec : compte les lignes où la conversion a échoué. C'est la version SQL du isna().sum() de pandas.	Confondre WHERE x IS NULL (les échecs) et WHERE x = '' (les cellules vides d'origine). Sur ce fichier, les 114 montants texte sont non vides mais non convertibles — il faut bien IS NULL après TRY_CAST.
LEFT JOIN	Jointure qui conserve toutes les lignes de la table de gauche, et met NULL quand la clé manque à droite. C'est l'écriture SQL du LeftOuter de Power Query.	Faire INNER JOIN par défaut : les 10 clients inconnus de mars disparaissent silencieusement (mesure m05_filrouge_mars_clients_inconnus), et le total n'est plus celui du fichier d'entrée.
Clé de jointure — Join key	Le ou les colonnes sur lesquelles on joint. La clé complète (id_client × annee × mois) renvoie 101 lignes enrichies sur mars ; la clé partielle (id_client seul) en renvoie 3 978 (≈ 39 × trop).	Joindre sur la clé partielle « pour simplifier » : c'est l'erreur la plus coûteuse du module.
GROUP BY	L'opération d'agrégation : regroupe les lignes par valeur de clé et applique une fonction d'agrégation (SUM, COUNT, AVG, MIN, MAX).	Oublier une colonne dans le SELECT : DuckDB renvoie une erreur (« column must appear in GROUP BY »).
Pivot en SQL — Pivot in SQL	L'écriture SQL du pivot est SUM(CASE WHEN colonne = 'valeur' THEN mesure ELSE 0 END) AS valeur. C'est verbeux, mais c'est universel .	Utiliser PIVOT (PostgreSQL) ou PIVOT (DuckDB ≥ 0.9) — c'est plus court, mais moins portable . La règle du module est toujours la forme SUM(CASE WHEN ...).
COPY ... TO	L'export DuckDB vers un fichier CSV ou Parquet. Plus rapide que SELECT ... FROM ... puis écriture ligne par ligne.	Oublier le format (COPY ... TO 'mars2025.parquet' (FORMAT PARQUET)) : DuckDB exporte en CSV par défaut, et le fichier n'a pas l'extension demandée.

Français — English	Définition simple	Piège à éviter
EXTRACT(MONTH FROM ...)	Extrait le mois (1-12) d'une date. C'est la détection des transpositions jour/mois : EXTRACT(MONTH FROM date_vente) <> mois.	Confondre MONTH et MONTHS (DuckDB n'a pas MONTHS — c'est MONTH).

5. Cours approfondi — la table des 10 opérations en SQL DuckDB

Cette section reprend la table du C01 (§5) et l'applique **exclusivement à SQL DuckDB 1,5,5**, avec un détail suffisant pour qu'un apprenant puisse reproduire la requête sans aide. Les blocs SQL publiés ici sont **exécutés** dans l'atelier — chaque chiffre cité est mesuré sur le fichier livré.

5.1 Session en mémoire et chargement du CSV

```
-- session en mémoire : on crée la connexion dans Python, ou via le CLI `duckdb`
-- (les deux sont équivalents ; le CLI n'est pas utilisé dans cet atelier)
.read_csv_auto '01_socle_donnees/data/brut/ventes_brutes.csv' AS ventes_brutes;

-- vérification
SELECT COUNT(*) FROM ventes_brutes; -- 243 360 lignes
DESCRIBE ventes_brutes;              -- 20 colonnes ; montant_ttc est en VARCHAR (texte)
```

- La **session en mémoire** est créée par `duckdb.connect(":memory:")` en Python, ou par `duckdb` en CLI. Les tables vivent dans la RAM ; elles disparaissent à la fin du processus.
- `read_csv_auto` infère les types : `id_vente` est en `BIGINT`, `date_vente` en `DATE`, mais `montant_ttc` est en `VARCHAR (texte)` — parce que la colonne contient des espaces-milliers et le suffixe « FCFA ». C'est le **comportement attendu** : on ne peut pas deviner le type.
- La **vérification** (`COUNT(*)`, `DESCRIBE`) est la première discipline : elle confirme que le chargement a bien produit 243 360 lignes et 20 colonnes.

5.2 Étape 1 — Sélectionner / renommer les colonnes

```
CREATE OR REPLACE TABLE v1_selection AS
SELECT
    id_vente,
    date_vente,
    id_ticket,
    id_produit,
    quantite,
    montant_ttc,          -- conservé en VARCHAR pour l'instant
    taux_remise,
    id_client,
    annee,
    mois,
    heure
FROM ventes_brutes;
```

- `CREATE OR REPLACE TABLE ... AS ...` crée une nouvelle table à partir de la requête. Le `OR REPLACE` permet de ré-exécuter sans erreur si la table existe déjà.
- Le **renommage** se fait dans le `SELECT` (alias `AS nouveau_nom`) ou via `AS` après le nom de colonne. DuckDB ne supporte pas la syntaxe `RENAME COLUMN` dans cette position — il faut recréer la table.

5.3 Étape 2 — Convertir le type (avec compteur d'échec)

```
CREATE OR REPLACE TABLE v2_conversion AS
SELECT
  *,
  TRY_CAST(
    REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '')
    AS BIGINT
  ) AS montant_ttc_n
FROM v1_selection;

-- compteur d'échec
SELECT
  'conversion_montant_ttc' AS etape,
  COUNT(*) AS echecs
FROM v2_conversion
WHERE TRY_CAST(REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '') AS BIGINT) IS NULL
  AND montant_ttc IS NOT NULL
  AND montant_ttc != '';
-- sur le brut complet : 4 211 échecs (les 1 710 montants texte de la fenêtre + leurs copies +
-- ressaisies). Sur mars 2025 : 114 (mesure m05_filrouge_mars_montants_texte).
```

- **TRY_CAST** est la conversion tolérante : en cas d'échec, la cellule devient `NULL` au lieu de faire échouer toute la requête. L'alternative stricte est `CAST` (qui fait échouer toute la ligne).
- Le **compteur d'échec** est crucial : c'est la **différence** entre une conversion traçable et une conversion silencieuse. Sans compteur, on additionne du texte et on perd 1,5 % du CA sans s'en apercevoir.
- **Piège classique** : `CAST` au lieu de `TRY_CAST`. Si on écrit `CAST(... AS BIGINT)` et qu'une cellule contient « 1 200 FCFA » (avant retrait du suffixe), DuckDB renvoie une erreur et la requête s'arrête. La règle est toujours `TRY_CAST` + compteur.

5.4 Étape 3 — Nettoyer le texte

```
CREATE OR REPLACE TABLE v3_nettoyage AS
SELECT
  *,
  TRIM(REPLACE(montant_ttc, ' FCFA', '')) AS montant_ttc_clean
FROM v2_conversion;

-- re-conversion après nettoyage (le retrait des espaces résiduels)
CREATE OR REPLACE TABLE v3b_conversion AS
SELECT
  *,
  TRY_CAST(REPLACE(montant_ttc_clean, ' ', '') AS BIGINT) AS montant_ttc_n
FROM v3_nettoyage;

-- compteur après nettoyage (devrait être plus petit)
SELECT COUNT(*) FROM v3b_conversion WHERE montant_ttc_n IS NULL AND montant_ttc != '';
-- sur mars 2025 : 0 (mesure m05_filrouge_mars_montants_texte_apres_conversion)
-- → le nettoyage marche : 114 lignes « ... FCFA » deviennent 114 entiers
```

- **Deux étapes de conversion** (avant et après nettoyage) sont **volontaires** : la première trace l'échec, la seconde confirme que le nettoyage a marché.
- L'ordre canonique est : **nettoyer le texte avant de convertir en nombre** (règle §3 du plan M05, règle 2). Si on convertit avant de nettoyer, le compteur d'échec reste à 114 et on ne sait pas si le nettoyage a marché.
- **TRIM** retire les espaces en début/fin de chaîne. C'est l'équivalent SQL du `.str.strip()` de pandas.

5.5 Étape 4 — Filtrer les lignes (la coupe à la date)

```
CREATE OR REPLACE TABLE v4_coupe AS
SELECT *
FROM v3b_conversion
WHERE date_vente BETWEEN DATE '2025-03-01' AND DATE '2025-03-31';

SELECT COUNT(*) FROM v4_coupe; -- 6 884 (mesure m05_filrouge_mars_lignes)
```

- **BETWEEN** est inclusif aux deux bornes : `BETWEEN '2025-03-01' AND '2025-03-31'` inclut le 1^{er} et le 31 mars. C'est le comportement souhaité pour les coupes mensuelles.
- **DATE '2025-03-01'** est un littéral de type date. DuckDB ne confond pas avec `TIMESTAMP` ou `VAR-CHAR` : le `DATE` est explicite.
- **Piège classique** : utiliser `BETWEEN '2025-03-01' AND '2025-03-31'` avec `date_vente` en `VARCHAR` : DuckDB convertit implicitement, mais l'index n'est pas utilisé et la requête est lente. La règle est toujours typer `date_vente` en `DATE` avant de filtrer.

5.6 Étape 5 — Trier

```
CREATE OR REPLACE TABLE v5_tri AS
SELECT *
FROM v4_coupe
ORDER BY date_vente, id_vente;
```

- L'**ordre de tri** est important pour le dédoublonnage (§5.7) : `ROW_NUMBER() OVER (... ORDER BY id_vente)` garde la première occurrence selon l'ordre du tri.
- Le tri **n'est pas obligatoire** en SQL : DuckDB peut dédoubler sans tri préalable si la fonction de rang est spécifiée correctement. Mais **trier rend le résultat déterministe** : sans tri, DuckDB peut renvoyer les lignes dans un ordre différent à chaque exécution, et l'empreinte sha256 change.

5.7 Étape 6 — Dédoubler (par rang)

```
CREATE OR REPLACE TABLE v6_dedup AS
WITH cle AS (
  SELECT
    *,
    ROW_NUMBER() OVER (
      PARTITION BY id_ticket, id_produit, quantite, montant_ttc_n, heure
      ORDER BY id_vente
    ) AS rn
  FROM v5_tri
)
SELECT * EXCLUDE rn
FROM cle
WHERE rn = 1;

SELECT COUNT(*) FROM v6_dedup; -- 6 798 (6 884 - 86 ; mesure m05_filrouge_mars_doublons)
```

- **PARTITION BY id_ticket, id_produit, quantite, montant_ttc_n, heure** est la **clé métier 5-colonnes** : c'est elle qui dédoublonne sans manger les 6 répétitions intra-ticket légitimes (mesure `m05_repetitions_legitimes_brut`).
- **ORDER BY id_vente** dans la fonction `ROW_NUMBER()` : on garde le plus petit `id_vente` (la ligne d'origine), pas la copie.
- **SELECT * EXCLUDE rn** : DuckDB 1,5,5 supporte la syntaxe `EXCLUDE` (depuis la 0,9). C'est plus lisible que de lister les 22 colonnes à la main.
- **Piège classique** : oublier `WHERE rn = 1` : la requête renvoie toutes les lignes, et le dédoublonnage n'a pas eu lieu.

5.8 Étape 7 — Colonne calculée (la transposition jour/mois)

```
CREATE OR REPLACE TABLE v7_repare AS
SELECT
  CASE
    WHEN EXTRACT(MONTH FROM date_vente) <> mois
    THEN STRPTIME(
      STRftime(date_vente, '%Y-%m-%d'),
      -- on échange jour et mois dans la chaîne ISO
      '%Y-' || LPAD(EXTRACT(DAY FROM date_vente)::VARCHAR, 2, '0')
        || '-' || LPAD(EXTRACT(MONTH FROM date_vente)::VARCHAR, 2, '0')
    )::DATE
    ELSE date_vente
  END AS date_vente_reparee,
  * EXCLUDE (date_vente)
FROM v6_dedup;

-- vérification : plus de date hors de mars
SELECT COUNT(*) FROM v7_repare
WHERE EXTRACT(MONTH FROM date_vente_reparee) <> 3;
-- doit être 0 (sinon, il reste une transposition non réparée)
```

- **Pourquoi STRftime puis STRPTIME** : DuckDB ne supporte pas nativement la transposition jour/mois sur une DATE. On formate la date en chaîne ISO ('%Y-%m-%d'), on échange le jour et le mois dans la chaîne, et on re-parse.
- **LPAD(..., 2, '0')** garantit que le jour et le mois sont sur 2 chiffres (le 7 mars devient '07', pas '7').
- **Piège classique** : oublier le ELSE date_vente : toutes les dates sont modifiées, y compris celles qui sont correctes.

5.9 Étape 8 — Fusionner (la jointure aux remises)

```
CREATE OR REPLACE TABLE v8_enrichi AS
WITH remises_dedup AS (
  SELECT DISTINCT id_client, annee, mois, remise_consentie_montant
  FROM read_xlsx('01_socle_donnees/data/brut/remises_manuelles.xlsx',
    sheet='Feuill1')
)
SELECT
  v.*,
  r.remise_consentie_montant
FROM v7_repare v
LEFT JOIN remises_dedup r
  ON v.id_client = r.id_client
  AND v.annee = r.annee
  AND v.mois = r.mois;

-- vérifications
SELECT COUNT(*) FROM v8_enrichi WHERE remise_consentie_montant IS NOT NULL;
-- 101 (mesure m05_filrouge_mars_lignes_enrichies)

-- la jointure naïve, pour comparer :
SELECT COUNT(*) FROM v8_enrichi v
INNER JOIN remises_dedup r ON v.id_client = r.id_client;
-- 3 978 (mesure m05_filrouge_mars_jointure_naive, ≈ 39 × trop)
```

- La **clé complète** (id_client × annee × mois) est l'unique protection contre la jointure naïve. C'est la leçon du module (règle 4 du §3 du plan M05).
- **LEFT JOIN** (et non INNER JOIN) : on garde toutes les lignes de v7_repare, et on met NULL pour les clients sans remise. Les 10 clients inconnus (mesure m05_filrouge_mars_clients_inconnus) restent dans la table.

- La **jointure naïve** (INNER JOIN sur `id_client` seul) est donnée **pour comparer** : sans clé complète, on multiplie les lignes par le nombre de mois couverts par le client. Sur mars : **3 978 lignes au lieu de 101** — sans message d’erreur.

5.10 Étape 9 — Agréger (CA par catégorie)

```
CREATE OR REPLACE TABLE v9_agregat AS
SELECT
  SUBSTRING(CAST(id_produit AS VARCHAR), 1, 1) AS cat_proxy,
  SUM(montant_ttc_n) AS ca,
  SUM(quantite) AS quantite_totale,
  COUNT(*) AS nb_lignes
FROM v8_enrichi
GROUP BY cat_proxy
ORDER BY ca DESC;

SELECT * FROM v9_agregat;
-- 9 lignes (mesure m05_brut_pivot_lignes sur le brut complet, 9 sur mars)
```

- `SUBSTRING(CAST(... AS VARCHAR), 1, 1)` est le proxy catégorie (la 1^{re} lettre de `id_produit`). C’est l’équivalent SQL du `.str[0]` de pandas.
- `GROUP BY cat_proxy` : DuckDB refuse les colonnes agrégées dans le `SELECT` qui ne sont pas dans le `GROUP BY`. C’est la garantie que l’agrégation est cohérente.

5.11 Étape 10 — Pivoter / dépivoter en SQL

Le pivot en SQL est **verbeux** mais universel :

```
CREATE OR REPLACE TABLE v10_pivot AS
SELECT
  mois,
  SUM(CASE WHEN cat_proxy = '1' THEN ca ELSE 0 END) AS cat_1,
  SUM(CASE WHEN cat_proxy = '2' THEN ca ELSE 0 END) AS cat_2,
  SUM(CASE WHEN cat_proxy = '3' THEN ca ELSE 0 END) AS cat_3,
  SUM(CASE WHEN cat_proxy = '4' THEN ca ELSE 0 END) AS cat_4,
  SUM(CASE WHEN cat_proxy = '5' THEN ca ELSE 0 END) AS cat_5,
  SUM(CASE WHEN cat_proxy = '6' THEN ca ELSE 0 END) AS cat_6,
  SUM(CASE WHEN cat_proxy = '7' THEN ca ELSE 0 END) AS cat_7,
  SUM(CASE WHEN cat_proxy = '8' THEN ca ELSE 0 END) AS cat_8,
  SUM(CASE WHEN cat_proxy = '9' THEN ca ELSE 0 END) AS cat_9
FROM v9_agregat
GROUP BY mois
ORDER BY mois;
```

- `SUM(CASE WHEN ... THEN ... ELSE 0 END)` est l’écriture **universelle** du pivot. Elle marche dans PostgreSQL, BigQuery, Snowflake, Redshift, DuckDB. C’est la forme à apprendre par cœur.
- L’alternative `PIVOT ...` (DuckDB $\geq 0,9$) est plus courte mais **moins portable** : la règle du module est toujours la forme `SUM(CASE WHEN ...)`.

Le dépivot (l’inverse) :

```
CREATE OR REPLACE TABLE v10_depivot AS
SELECT mois, cat_proxy, ca
FROM v10_pivot
UNPIVOT (
  ca FOR cat_proxy IN (cat_1, cat_2, cat_3, cat_4, cat_5, cat_6, cat_7, cat_8, cat_9)
);
```

- `UNPIVOT` est une syntaxe DuckDB (et PostgreSQL, Oracle). La clause `IN (...)` liste les colonnes à dépivoter ; la clause `FOR cat_proxy IN (...)` donne le nom de la nouvelle colonne de catégories.

- Le **dépivot** du pivot précédent rend $9 \times 12 = 108$ lignes (mesure `m05_brut_depivot_lignes` sur le brut complet).

5.12 Étape 11 — Exporter la table propre

```
-- export CSV
COPY (SELECT * FROM v10_depivot) TO '01_socle_donnees/data/projection/mars2025.csv' (HEADER);

-- export Parquet (plus compact, typé, recommandé pour les tables > 100 000 lignes)
COPY (SELECT * FROM v10_depivot) TO '01_socle_donnees/data/projection/mars2025.parquet' (FORMAT
PARQUET);
```

- **COPY (SELECT ... FROM ...) TO '...' (HEADER)** est l'export DuckDB. Le **HEADER** ajoute l'en-tête.
- **FORMAT PARQUET** est obligatoire pour le format Parquet (DuckDB n'infère pas le format).
- Le **format Parquet** est recommandé pour les tables > 100 000 lignes : il est 5-10 × plus compact que CSV, et il conserve les types (DuckDB n'a pas besoin de réinférer les types à la lecture).

5.13 La session en mémoire, persistée seulement à l'export

La règle §1.3 du plan M05 est **claire** : pas de fichier `.duckdb` persisté. La session est en mémoire ; seule la **sortie** (CSV ou Parquet) est persistée. C'est ce qui rend SQL DuckDB reproductible : le pipeline est dans le code SQL, pas dans un fichier de base.

```
# en Python (le contrôleur controle_sql.py utilise cette forme)
import duckdb
con = duckdb.connect(":memory:")
con.execute("""
    .read_csv_auto '01_socle_donnees/data/brut/ventes_brutes.csv' AS ventes_brutes
""") # ATTENTION : cette syntaxe est celle du CLI, pas de Python
# en Python, on fait :
con.execute("""
    CREATE OR REPLACE TABLE ventes_brutes AS
    SELECT * FROM read_csv_auto('01_socle_donnees/data/brut/ventes_brutes.csv')
""")
```

- En **Python**, `duckdb.connect(":memory:")` crée la session. Les commandes `READ_CSV_AUTO` doivent être à l'intérieur d'un `CREATE TABLE AS` (le CLI accepte la syntaxe `.read_csv_auto ... AS ventes_brutes` qui n'est pas valide en Python).
- La session meurt à la fin du `with duckdb.connect(":memory:") as con:`. Les tables sont perdues.

5.14 Le diagnostic en 4 questions (le « Health Check » SQL)

Quand une requête ne fait pas ce qu'elle devrait, **4 questions** dans l'ordre :

1. **Le type est-il bon ?** `DESCRIBE ventes_brutes` — `montant_ttc` doit être en `VARCHAR` après le `read_csv_auto`, et `montant_ttc_n` doit être en `BIGINT` après la conversion.
2. **Le compteur d'échec est-il cohérent ?** `SELECT COUNT(*) FROM v2_conversion WHERE TRY_CAST(...) IS NULL AND montant_ttc != ''` — sur le brut, doit rendre 4 211 ; sur mars après nettoyage, doit rendre 0.
3. **La jointure utilise-t-elle la clé complète ?** Le `ON` doit mentionner **3 colonnes** (`id_client`, `annee`, `mois`) — pas 1.
4. **L'agrégation est-elle cohérente ?** `SELECT COUNT(*) FROM v9_agregat` — doit rendre 9 (le proxy catégorie a 9 valeurs).

Conseil professionnel. Le **Health Check** en 4 questions est le premier réflexe à installer quand une requête SQL ne fait pas ce qu'elle devrait. Type, compteur, jointure, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes.

6. Exemple concret — la requête `mars2025.sql` complète

La requête **complète** du fil rouge mars 2025. C'est l'équivalent SQL de la requête Power Query de C02 et du notebook pandas de C01 §6.3. Les 6 blocs `WITH` (ou `CREATE TABLE AS`) sont commentés.

```
-- === Session en mémoire (règle §1.3 du plan M05) ===
-- (créée par duckdb.connect(":memory:") en Python, ou par `duckdb` en CLI)

-- === Chargement du brut ===
CREATE OR REPLACE TABLE ventes_brutes AS
SELECT * FROM read_csv_auto('01_socle_donnees/data/brut/ventes_brutes.csv');

-- === Étape 1 : Sélectionner les colonnes utiles ===
CREATE OR REPLACE TABLE v1 AS
SELECT
    id_vente, date_vente, id_ticket, id_produit, quantite, montant_ttc,
    taux_remise, id_client, annee, mois, heure
FROM ventes_brutes;

-- === Étape 2 : Conversion de montant_ttc (TRY_CAST + compteur) ===
CREATE OR REPLACE TABLE v2 AS
SELECT
    *,
    TRY_CAST(
        REPLACE(REPLACE(montant_ttc, ' FCFA', ''), ' ', '')
        AS BIGINT
    ) AS montant_ttc_n
FROM v1;

-- compteur d'échec (à logger en sortie)
-- sur mars 2025 : 114 (mesure m05_filrouge_mars_montants_texte)

-- === Étape 3 : Nettoyage du texte ===
CREATE OR REPLACE TABLE v3 AS
SELECT
    *,
    TRIM(REPLACE(montant_ttc, ' FCFA', '')) AS montant_ttc_clean
FROM v2;

-- re-conversion après nettoyage
CREATE OR REPLACE TABLE v3b AS
SELECT
    *,
    TRY_CAST(REPLACE(montant_ttc_clean, ' ', '') AS BIGINT) AS montant_ttc_n2
FROM v3;

-- === Étape 4 : Coupe à la date ===
CREATE OR REPLACE TABLE v4 AS
SELECT *
FROM v3b
WHERE date_vente BETWEEN DATE '2025-03-01' AND DATE '2025-03-31';

-- 6 884 lignes (mesure m05_filrouge_mars_lignes)

-- === Étape 5 : Tri ===
CREATE OR REPLACE TABLE v5 AS
SELECT * FROM v4 ORDER BY date_vente, id_vente;

-- === Étape 6 : Dédoublonnage par rang (clé métier 5-colonnes) ===
CREATE OR REPLACE TABLE v6 AS
WITH cle AS (
    SELECT
        *,
        ROW_NUMBER() OVER (
            PARTITION BY id_ticket, id_produit, quantite, montant_ttc_n2, heure
            ORDER BY id_vente
        ) AS rn
    FROM v5
)
SELECT * EXCLUDE rn FROM cle WHERE rn = 1;

-- 6 798 lignes (6 884 - 86 ; mesure m05_filrouge_mars_doublons)

-- === Étape 7 : Réparation des dates transposées ===
CREATE OR REPLACE TABLE v7 AS
SELECT
```

```

CASE
  WHEN EXTRACT(MONTH FROM date_vente) <> mois
  THEN STRPTIME(
    STRftime(date_vente, '%Y-%m-%d'),
    '%Y-' || LPAD(EXTRACT(DAY FROM date_vente)::VARCHAR, 2, '0')
    || '-' || LPAD(EXTRACT(MONTH FROM date_vente)::VARCHAR, 2, '0')
  )::DATE
  ELSE date_vente
END AS date_vente_reparee,
* EXCLUDE (date_vente)
FROM v6;

-- 23 dates transposées sur mars (mesure m05_filrouge_mars_dates_transposees)

-- === Étape 8 : Jointure aux remises (clé complète) ===
CREATE OR REPLACE TABLE v8 AS
WITH remises_dedup AS (
  SELECT DISTINCT id_client, annee, mois, remise_consentie_montant
  FROM read_xlsx('01_socle_donnees/data/brut/remises_manuelles.xlsx', sheet='Feuil1')
)
SELECT
  v.*,
  r.remise_consentie_montant
FROM v7 v
LEFT JOIN remises_dedup r
  ON v.id_client = r.id_client
  AND v.annee = r.annee
  AND v.mois = r.mois;

-- 101 lignes enrichies (mesure m05_filrouge_mars_lignes_enrichies)
-- la jointure naïve (INNER JOIN sur id_client seul) rend 3 978 lignes
(m05_filrouge_mars_jointure_naive)

-- === Étape 9 : Agrégation par catégorie (proxy) ===
CREATE OR REPLACE TABLE v9 AS
SELECT
  SUBSTRING(CAST(id_produit AS VARCHAR), 1, 1) AS cat_proxy,
  SUM(montant_ttc_n2) AS ca,
  SUM(quantite) AS quantite_totale,
  COUNT(*) AS nb_lignes
FROM v8
GROUP BY cat_proxy
ORDER BY ca DESC;

-- === Étape 10 : Pivot ===
CREATE OR REPLACE TABLE v10 AS
SELECT
  'mars2025' AS mois,
  SUM(CASE WHEN cat_proxy = '1' THEN ca ELSE 0 END) AS cat_1,
  SUM(CASE WHEN cat_proxy = '2' THEN ca ELSE 0 END) AS cat_2,
  SUM(CASE WHEN cat_proxy = '3' THEN ca ELSE 0 END) AS cat_3,
  SUM(CASE WHEN cat_proxy = '4' THEN ca ELSE 0 END) AS cat_4,
  SUM(CASE WHEN cat_proxy = '5' THEN ca ELSE 0 END) AS cat_5,
  SUM(CASE WHEN cat_proxy = '6' THEN ca ELSE 0 END) AS cat_6,
  SUM(CASE WHEN cat_proxy = '7' THEN ca ELSE 0 END) AS cat_7,
  SUM(CASE WHEN cat_proxy = '8' THEN ca ELSE 0 END) AS cat_8,
  SUM(CASE WHEN cat_proxy = '9' THEN ca ELSE 0 END) AS cat_9
FROM v9;

-- === Étape 11 : Export (CSV + Parquet) ===
COPY (SELECT * FROM v10) TO '01_socle_donnees/data/projection/mars2025_pivot.csv' (HEADER);
COPY (SELECT * FROM v10) TO '01_socle_donnees/data/projection/mars2025_pivot.parquet' (FORMAT
PARQUET);

```

Verdict attendu pour mars 2025 : - 6 884 lignes avant dédoublement (mesure `m05_filrouge_mars_lignes`) - 6 798 lignes après dédoublement (6 884 - 86 ; mesure `m05_filrouge_mars_doublons`) - Total montant_ttc = 464 096 003 FCFA (mesure `m05_filrouge_mars_total_ttc`) - 101 lignes enrichies (mesure `m05_filrouge_mars_lignes_enrichies`) - 23 dates transposées réparées (mesure `m05_filrouge_mars_dates_transposees`) - Compteur d'erreur

de conversion après nettoyage = 0 (114 lignes avant nettoyage ; mesure `m05_fil-rouge_mars_montants_texte`)

Définition. Une **CTE** (`WITH ... AS ...`) est une sous-requête nommée, réutilisable dans la requête principale. DuckDB optimise les CTE par inline (la sous-requête est recopiée là où elle est utilisée), ce qui rend les CTE aussi performantes que les sous-requêtes classiques. La règle du module est un `WITH` par étape conceptuelle, nommé en français.

Définition. Le **EXCLUDE** est une clause DuckDB 1,5,5 (depuis la 0,9) qui permet de retirer des colonnes du `SELECT` : `SELECT * EXCLUDE (rn, date_vente)` retire `rn` et `date_vente`. C'est plus lisible que de lister les 22 colonnes manuellement. La règle du module : toujours utiliser `EXCLUDE` plutôt que de lister les colonnes une par une.

Définition. Le **LPAD** (left-pad) est la fonction SQL DuckDB qui pad une chaîne à gauche avec un caractère donné : `LPAD('7', 2, '0')` rend `'07'`. C'est nécessaire pour la transposition jour/mois : sans `LPAD`, le 7 mars deviendrait `'2025-7-3'` au lieu de `'2025-07-03'`, et la date ne serait pas reparsable.

Définition. Le **session variable** est une variable de session DuckDB, créée par `SET nom = valeur`. Elle est accessible dans toutes les requêtes suivantes de la même session. C'est l'équivalent SQL du paramètre Power Query ou de la constante en haut d'un notebook pandas.

7. Démonstration pas à pas — le verdict : un fichier, trois moteurs, une table

Le **contrôle** que la requête SQL DuckDB produit la même table que la requête Power Query du C02 et le notebook pandas du C01 §6.3 est la **clé de voûte** du module. La méthode est la même que dans le C01 §7 : sérialiser la sortie au format canonique, trier par `id_vente`, hasher en sha256.

7.1 La sérialisation canonique en SQL

```
-- la table propre est v10 (le pivot final)
-- pour l'empreinte, on repart de v8 (avant agrégation) – c'est la table de détail
COPY (
    SELECT * FROM v8 ORDER BY id_vente
) TO '01_socle_donnees/data/projection/mars2025_v8.csv' (HEADER);
```

- La **sérialisation canonique** est la même que dans C01 §7.1 : `montant_ttc` en entier, dates en ISO, NaN/NULL sérialisés en chaîne vide, lignes triées par `id_vente`.
- L'**export CSV** est fait par `COPY (SELECT ... FROM ... ORDER BY ...) TO ...` — DuckDB trie à l'export, et le fichier est sérialisé dans l'ordre canonique.

7.2 Le calcul de l’empreinte en Python

```
# bloc publié dans le contrôleur controle_sql.py (extrait)
import duckdb
import hashlib

con = duckdb.connect(":memory:")
con.execute("""
    CREATE OR REPLACE TABLE v AS
    SELECT * FROM read_csv_auto('01_socle_donnees/data/projection/mars2025_v8.csv')
""")

lignes = []
for r in con.execute("SELECT * FROM v ORDER BY id_vente").fetchall():
    lignes.append("|".join("" if v is None else str(v) for v in r))

empreinte = hashlib.sha256("\n".join(lignes).encode("utf-8")).hexdigest()
print("empreinte DuckDB :", empreinte)
# attendu : la même empreinte que pandas et Power Query (la table est identique)
```

- Le **calcul d’empreinte** en Python est plus simple qu’en DuckDB pur, parce que Python a le contrôle fin sur le format de sérialisation.
- L’empreinte attendue est **identique** à celle de pandas et Power Query — c’est la **preuve** que les trois moteurs produisent la même table.

7.3 Le verdict du projet M05.P (la mesure d’ouverture)

Sur le **dossier projet M05.P** (03_exercices/dossier_M05/, recalculé en DuckDB 1,5,5 et vérifié en pandas — les deux moteurs ont la même empreinte) : - 120 000 lignes finales (mesure `m05p_lignes_final`) - Total montant_ttc = 7 145 910 735 FCFA (mesure `m05p_total_ttc_final`) - Empreinte sha256 = 7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465 (mesure `m05p_empreinte_sha256`)

À retenir. Le verdict du projet M05.P est mesuré en pandas ET en DuckDB, et les deux moteurs donnent la **même** empreinte sha256. C’est la **preuve** que le pipeline est portable et déterministe.

Attention. Le **compteur d’échec de conversion** peut rendre 0 après la conversion réussie, même si la conversion a échoué sur certaines lignes : c’est le cas si `TRY_CAST` est appelé sur la colonne nettoyée et non sur la colonne brute. La règle est toujours compter les échecs sur la **colonne brute** (avant nettoyage) ET sur la **colonne nettoyée** (après nettoyage) — la différence des deux compteurs est le nombre de lignes réparées par le nettoyage. Sur mars 2025 : 114 avant nettoyage (mesure `m05_filrouge_mars_montants_texte`), 0 après nettoyage (mesure `m05_filrouge_mars_montants_texte_apres_conversion`), soit 114 lignes réparées par le nettoyage.

Attention. DuckDB `read_csv_auto` est **plus tolérant** que `pd.read_csv` : il devine le séparateur (; ou ,), l’encodage (UTF-8 ou cp1252), et la présence d’un BOM. C’est un avantage pour le prototypage rapide, mais c’est un **risque** pour la production : si le CSV change de séparateur entre deux fichiers, DuckDB peut pivoter silencieusement. La règle est toujours spécifier le séparateur (`read_csv_auto(..., delim=',')`) pour les pipelines de production.

7.4 La rejouabilité : avril et mai 2025

Le **même** pipeline SQL, exécuté sur avril et mai 2025 (changement de la borne supérieure du `BETWEEN`) :

```
-- pour avril : WHERE date_vente BETWEEN DATE '2025-04-01' AND DATE '2025-04-30'
-- pour mai   : WHERE date_vente BETWEEN DATE '2025-05-01' AND DATE '2025-05-31'
```

Lignes attendues : 6 639 (avril, mesure `m05_filrouge_avril_lignes`) · 5 530 (mai, mesure `m05_filrouge_mai_lignes`).

8. Erreurs fréquentes

- **Utiliser `CAST` au lieu de `TRY_CAST`.** `CAST` fait échouer toute la ligne en cas d'erreur, ce qui peut produire des résultats incomplets (parfois 0 lignes, parfois N-1 lignes). La règle est toujours `TRY_CAST` + compteur d'échec.
- **Oublier le compteur d'échec.** Sans `COUNT(*) WHERE TRY_CAST(...) IS NULL AND ...`, on ne sait pas combien de lignes ont été perdues. Une conversion silencieuse est une conversion sans information.
- **Joindre sur la clé partielle** (`id_client` seul) au lieu de la clé complète (`id_client × annee × mois`). La jointure naïve rend **3 978 lignes** au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`). DuckDB ne dit rien — la requête tourne, le résultat est faux.
- **Faire `INNER JOIN` par défaut.** Les 10 clients inconnus de mars (mesure `m05_filrouge_mars_clients_inconnus`) disparaissent silencieusement, et le total n'est plus celui du fichier d'entrée.
- **Oublier `WHERE rn = 1` après le dédoublonnage par rang.** La requête renvoie toutes les lignes, et le dédoublonnage n'a pas eu lieu.
- **Oublier le `ORDER BY` dans la fonction `ROW_NUMBER() OVER (...)`.** Sans `ORDER BY`, DuckDB attribue les rangs dans un ordre arbitraire — la table résultante n'est pas déterministe, et l'empreinte sha256 change à chaque exécution.
- **Croire que `read_csv_auto` a fait la conversion.** `read_csv_auto` met `montant_ttc` en `VARCHAR` (texte) à cause du suffixe « FCFA ». La conversion en `BIGINT` est à refaire avec `TRY_CAST`.
- **Persister la session dans un fichier `.duckdb`** : c'est plus lourd, et le fichier se désynchronise de la source. Règle §1.3 du plan M05 : pas de base persistée.

À retenir. Les 5 erreurs qui coûtent le plus cher en SQL : (1) `CAST` au lieu de `TRY_CAST` ; (2) absence de compteur d'échec ; (3) jointure sur clé partielle ; (4) `INNER JOIN` par défaut ; (5) `WHERE rn = 1` oublié après le dédoublonnage par rang. La règle unique : toujours expliciter l'option par défaut.

9. Bonnes pratiques professionnelles

- **Chaque table temporaire a un nom, en toutes lettres.** `v8_enrichi` plutôt que `t1`. Sans nom, la requête est illisible et **personne ne peut la maintenir** (règle R7 du §3 du plan M05).
- **Le compteur d'échec n'est jamais en commentaire.** C'est l'étape `SELECT 'conversion', COUNT(*) FROM ... WHERE TRY_CAST(...) IS NULL` qui rend 114 sur mars avant nettoyage, 0 après. Sans compteur, on ne sait pas si la conversion est complète.
- **La session est en mémoire, la sortie est persistée.** La session DuckDB meurt à la fin du `with con:`. La sortie (CSV ou Parquet) est la table propre, transmissible. Règle §1.3 du plan M05.
- **Le format Parquet est préféré au CSV** pour les tables > 100 000 lignes : il est 5-10 × plus compact, et il conserve les types.
- **La jointure utilise la clé complète**, documentée dans un commentaire au-dessus du `JOIN`. C'est la règle 4 du §3 du plan M05.

- **Le dédoublement utilise la méthode `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`** — c'est la forme universellement supportée. `DISTINCT ON` n'existe qu'en PostgreSQL ; `QUALIFY` n'existe que dans BigQuery/Snowflake.
- **Le verdict est publié en clé `m05_*`**. Chaque chiffre cité dans la requête (6 884 lignes, total 464 096 003 FCFA, 114 montants texte, 86 doublons, 101 enrichies, 23 transpositions) provient de `chiffres_manuel.py`.

Conseil professionnel. Le **Health Check** en 4 questions du §5.14 est le premier réflexe à installer quand une requête SQL ne fait pas ce qu'elle devrait. Type, compteur, jointure, agrégation : 4 questions dans l'ordre, 90 % des bugs sont trouvés en moins de 5 minutes.

10. Exercice guidé — la requête `avril2025.sql` paramétrée (45 min, /10)

Objectif. Reprendre la requête `mars2025.sql` du §6 et la rendre **réellement paramétrée** sur le mois cible. Vérifier que la requête produit avril 2025 avec les bons chiffres.

Énoncé.

1. Créer la variable de session : `SET mois_debut = '2025-04-01'; SET mois_fin = '2025-04-30';`
2. Modifier la requête `mars2025.sql` pour utiliser ces variables (au lieu de coder `DATE '2025-03-01'` en dur).
3. Exécuter la requête et vérifier : **6 639 lignes** (mesure `m05_filrouge_avril_lignes`).
4. Changer les variables à `'2025-05-01' / '2025-05-31'`, ré-exécuter, vérifier : **5 530 lignes** (mesure `m05_filrouge_mai_lignes`).

Barème (/10).

Critère	Points
Les variables de session sont créées	2
La requête utilise les variables (pas de date en dur)	2
Pour avril : 6 639 lignes	2
Pour mai : 5 530 lignes	2
Compteur d'erreur de conversion affiché dans la requête (sa valeur n'est pas jugée, sa présence oui)	2
Total	10

Dans les faits. Cet exercice prend 35 minutes à un apprenant qui a déjà pratiqué le §6, et 1 h 15 à un apprenant qui découvre. La différence est presque entièrement sur l'étape **2** (utiliser les variables de session dans les littéraux de date) : DuckDB n'accepte pas les variables dans tous les contextes, et un apprenant qui n'a pas compris ça passe 30 minutes à chercher pourquoi `BETWEEN mois_debut AND mois_fin` échoue avec « syntax error at or near "mois_debut" ».

11. Exercices autonomes

- **Exercice 11.1 (15 min).** Dans la requête `mars2025.sql` du §6, remplacer l'étape `v2` (conversion) par une étape qui utilise `CAST` (au lieu de `TRY_CAST`). Exécuter la requête et noter l'erreur. C'est l'objet du piège n°1 du §8 : `CAST` fait échouer toute la requête.

- **Exercice 11.2 (30 min).** La requête `mars2025.sql` produit 11 tables temporaires (`v1` à `v10`). **Fusionner** les étapes `v1` et `v2` en une seule requête avec `TRY_CAST` dans le `SELECT`. Vérifier que le compteur d'échec rend toujours 114 sur mars.
- **Exercice 11.3 (45 min).** Reprendre la requête `mars2025.sql` et **ajouter** la jointure naïve (`INNER JOIN` sur `id_client` seul) **dans une table temporaire `v8_naive`**. Vérifier : **3 978 lignes** (mesure `m05_filrouge_mars_jointure_naive`), et comparer le total `montant_ttc` de `v8_naive` avec celui de `v8_enrichi` (la jointure naïve multi-compte les remises).
- **Exercice 11.4 (60 min, optionnel).** Mesurer le temps d'exécution de la requête `mars2025.sql` sur le brut complet (243 360 lignes). Répéter 5 fois, calculer la moyenne et l'écart-type. Comparer aux temps des requêtes Power Query (C02) et pandas (C04) sur le même fichier. **Étiqueter** le résultat comme « estimation, à vérifier sur un fichier plus gros ».

12. Correction détaillée

- **Exercice 11.1.** `CAST` au lieu de `TRY_CAST` fait échouer la requête dès la première cellule non convertible. Sur mars 2025, l'erreur est `Conversion Error: Could not convert string '1 200 FCFA' to int64`. La règle du module est toujours `TRY_CAST` + compteur d'échec.
- **Exercice 11.2.** La fusion des étapes `v1` et `v2` est possible mais rend la requête moins lisible. La règle du module est une étape par table temporaire, nommée en français — la lisibilité est prioritaire sur la concision.
- **Exercice 11.3.** La jointure naïve (`v8_naive`) rend bien **3 978 lignes** (mesure `m05_filrouge_mars_jointure_naive`). Le total `remise_consentie_montant` est **multiplié** par le nombre de mois couverts par le client — c'est l'erreur du débutant.
- **Exercice 11.4.** Les temps dépendent de la machine ; sur l'atelier de référence (DuckDB 1,5,5, machine virtuelle Linux 4 Go), les **mesures** publiées en C05 sont : 0,4 s SQL DuckDB (mars), ≈ 14 s SQL DuckDB (brut complet). Note : ces valeurs ne sont pas des clés `m05_*` ; elles sont étiquetées « estimation, à vérifier ».

13. Mini-projet M05.P3 — « La requête SQL du projet M05.P » (1 h 30)

Énoncé. Reprendre la requête `mars2025.sql` du §6 et l'appliquer au **vrai fichier du projet** (`03_exercices/dossier_M05/ventes_2023_2024.csv`, 121 720 lignes). Le **livrable** est un script SQL `projet2025.sql` qui : - Coupe à la date sur les **120 000 premières lignes** du fichier (la fenêtre du projet). - Applique toutes les étapes du §6. - Exporte la table finale en Parquet (`projet2025.parquet`). - Vérifie l'empreinte sha256 par rapport à l'ATTENDU du dossier (mesure `m05p empreinte_sha256 = 7cce2d0c...`).

Critères de réussite.

1. **120 000 lignes** dans la sortie (mesure `m05p_lignes_final`).
2. **Total `montant_ttc` = 7 145 910 735 FCFA** (mesure `m05p_total_ttc_final`).
3. **493 dates transposées réparées** (mesure `m05p_dates_transposees_reparees`).
4. **Compteur d'erreur de conversion affiché** dans la requête (sa valeur n'est pas jugée, sa présence oui).
5. **Empreinte sha256 identique** à l'ATTENDU (mesure `m05p empreinte_sha256`).
6. **Aucune valeur de `montant_ttc` négative** n'a été filtrée silencieusement (les retours sont conservés).

Barème (/10). 2 points par critère sauf le 5 (empreinte) qui vaut 2 points, le tout sommant 10.

14. Boîte à outils du chapitre

Boîte à outils. Sept objets, un seul but — reproduire la requête `mars2025.sql` dans DuckDB :

1. **La table des 10 opérations en SQL** (§5) — l'application SQL de la table du C01. À imprimer.
2. **La requête `mars2025.sql` complète** (§6) — les 11 tables temporaires, en SQL copiable.
3. **La session en mémoire** (§5.13) — `duckdb.connect(":memory:");`, sans fichier `.duckdb` persisté.
4. **Le dédoublonnage par rang** (§5.7) — `ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`, l'unique méthode universellement supportée.
5. **La jointure à la clé complète** (§5.9) — `LEFT JOIN ... ON id_client AND annee AND mois`, jamais sur `id_client` seul.
6. **Le compteur d'échec** (§5.3) — `COUNT(*) WHERE TRY_CAST(...) IS NULL AND ... ≠ ''`, après chaque conversion.
7. **Le verdict publié en clé `m05_*`** (§7) — chaque chiffre cité doit apparaître dans `chiffres_cites.json` ; sinon, c'est une promesse creuse.

15. Résumé du chapitre

- **SQL DuckDB** est l'outil de **transformation en volume** : 30 × plus rapide que pandas sur ce fichier (0,4 s vs 3,1 s sur 6 884 lignes, estimation à vérifier en C05), et il monte jusqu'à 10-50 millions de lignes sans suer.
- La **session en mémoire** (`duckdb.connect(":memory:");`) est ce qui rend SQL aussi léger que pandas. La règle §1.3 du plan M05 est pas de fichier `.duckdb` persisté.
- La **jointure à la clé complète** (`id_client × annee × mois`) est l'unique protection contre la jointure naïve (3 978 vs 101 sur mars, 41 716 vs 1 714 sur la fenêtre). DuckDB ne dit rien — la requête tourne, le résultat est faux.
- Le **dédoublonnage par rang** (`ROW_NUMBER() OVER (PARTITION BY ... ORDER BY ...)`) est la méthode universellement supportée (PostgreSQL, BigQuery, Snowflake, Redshift, DuckDB).
- Le **compteur d'échec** (`COUNT(*) WHERE TRY_CAST(...) IS NULL`) est la première discipline : DuckDB `read_csv_auto` est tolérant et avale les 1 710 montants texte sans un mot.
- Le **verdict** du projet M05.P : **empreinte sha256 = 7cce2d0c6253f739-faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465** (mesure `m05p Empreinte_sha256`), identique entre pandas et DuckDB. C'est la **preuve** que le pipeline est portable.

16. À retenir

À retenir. SQL DuckDB est un **moteur de transformation de tables** qui vit en mémoire. La session est jetable (règle §1.3 du plan M05) ; seule la sortie (CSV ou Parquet) est persistée. La requête paramétrée (par variables de session) est la clef de la robustesse au fichier du mois suivant. La jointure à la clé complète est l'unique protection contre l'erreur la plus coûteuse du module (3 978 lignes au lieu de 101 sur mars).

À retenir. Les 5 erreurs qui coûtent le plus cher en SQL : (1) `CAST` au lieu de `TRY_CAST` ; (2) absence de compteur d'échec ; (3) jointure sur clé partielle ; (4) `INNER JOIN` par défaut ; (5) `WHERE rn = 1` oublié après le dédoublonnage par rang. La règle unique : toujours expliciter l'option par défaut.

17. Évaluation formative (auto-correction, 8 min)

Question 1. Citez les 11 tables temporaires de la requête `mars2025.sql` du §6 dans l'ordre. (Réponse : §6.)

Question 2. Quelle est la différence entre `CAST` et `TRY_CAST` en SQL DuckDB ? Quand utilise-t-on chacun ? (Réponse : `CAST` est strict — il fait échouer toute la requête en cas d'erreur ; `TRY_CAST` est tolérant — il renvoie `NULL` en cas d'erreur. On utilise `TRY_CAST` sur des données sales, et `CAST` quand on est sûr du type.)

Question 3. Pourquoi la **jointure à la clé complète** est-elle critique ? (Réponse : parce que la jointure naïve sur `id_client` seul rend 3 978 lignes au lieu de 101 sur mars (mesure `m05_filrouge_mars_jointure_naive`) — sans message d'erreur. La règle du module est toujours la clé complète, et on la documente dans un commentaire au-dessus de la jointure.)

Question 4. Qu'est-ce que le **dédoublonnage par rang** ? Pourquoi est-il préférable à `DISTINCT ON` ou `QUALIFY` ? (Réponse : `ROW_NUMBER() OVER (PARTITION BY clé ORDER BY ...)` `WHERE rn = 1` attribue un rang à chaque ligne du groupe et garde la première. C'est universellement supporté (PostgreSQL, BigQuery, Snowflake, Redshift, DuckDB) ; `DISTINCT ON` n'existe qu'en PostgreSQL, `QUALIFY` n'existe que dans BigQuery/Snowflake.)

Question 5. Sur mars 2025, combien de dates transposées sont réparées par l'étape `v7` ? (Réponse : 23 lignes, mesure `m05_filrouge_mars_dates_transposees`.)

Question 6. Quelle est l'empreinte sha256 de la table `df_final` du projet M05.P ? Que prouve cette empreinte ? (Réponse : `7cce2d0c6253f739faf24c7ed205d0f12e6c81ef5f8b7fcd4826ad3bbd98a465` (mesure `m05p empreinte_sha256`). Elle prouve que la table produite par pandas est identique au bit près à celle produite par DuckDB — c'est la preuve que le pipeline est portable et déterministe.)

Question 7. Citez les 4 questions du Health Check SQL dans l'ordre. (Réponse : §5.14. (1) Le type est-il bon ? (2) Le compteur d'échec est-il cohérent ? (3) La jointure utilise-t-elle la clé complète ? (4) L'agrégation est-elle cohérente ?)